

SQL

BANCO DE DADOS I

PROF. DR. CARLOS ROBERTO VALÊNCIO

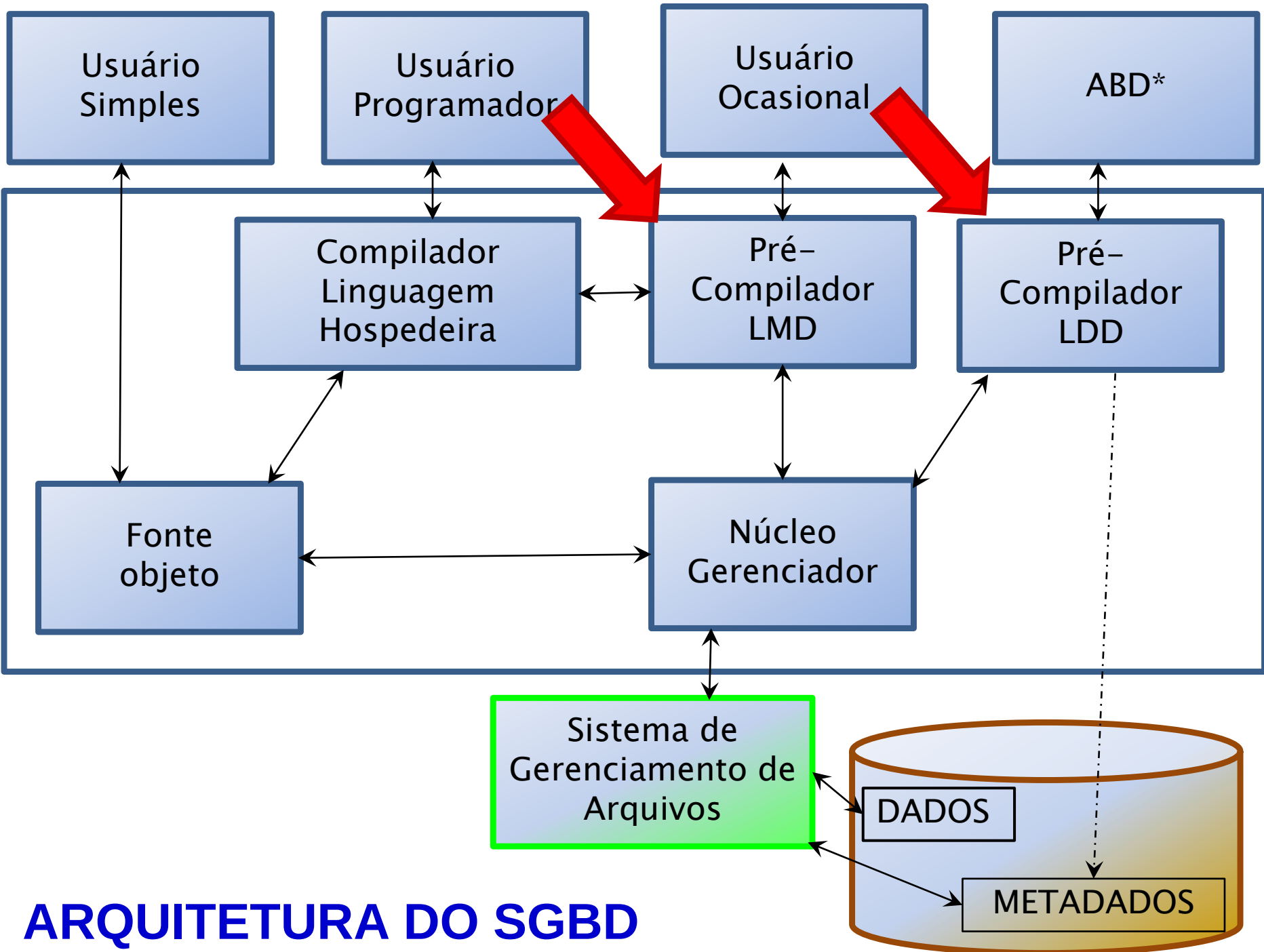
UNESP – São José do Rio Preto

Sumário

- **Capítulo 7 –SQL**
 - ▣ **7.1 Introdução**
 - ▣ **7.2 DDL**
 - ▣ **7.3 Consultas SQL**
 - ▣ **7.4 Funções de Agregação**
 - ▣ **7.5 Atualização de dados**

SQL

7.1 Introdução



Introdução

- ❑ SQL – “Structured Query Language” ou Linguagem de Consulta Estruturada.
- ❑ Padrão adotado para linguagens de consultas em Sistemas Gerenciadores de Banco de Dados Relacionais (SGBDR).
- ❑ Utilizada nos SGBDRs: DB2, Firebird, InterBase, Microsoft SQL Server, MySQL, Oracle, PostgreSQL, Microsoft Access, Sybase etc.

Introdução

- Com o uso de cláusulas SQL é possível realizar tarefas como definir a estrutura das tabelas; manipular os dados armazenados – inserir, eliminar e consulta-los; entre outras.
- Os SGBDRs implementam variações do padrão SQL, incluindo particularidades e funcionalidades adicionais.
- Os componentes básicos da SQL geralmente são comuns a todos os SGBDRs

Introdução

- Tópicos a seguir:
 - ▣ Linguagem de definição de dados (DDL)
 - ▣ Linguagem de manipulação de dados (DML)
 - ▣ Integridade

SQL

7.2 DDL

DDL

- O conjunto de relações da base de dados devem ser definidos através do uso da linguagem DDL.
- Define-se a estrutura, os tipos de atributos associados, restrições de integridade, regras de autorização para cada relação e outras informações relevantes.

Projeto lógico

- ❑ Cliente(RG, nome_cli, ano_nasc, cidade_cli, sexo)
 - ❑ Agencia(cod_ag, nome_ag, cidade_ag, ativo)
 - ❑ Depósito(num_dep, RG, saldo, cod_ag)
 - ❑ Empréstimo(num_empr, RG, valor, cod_ag)
-
- ```
graph TD; C1[Cliente] -- RG --> D1[Depósito]; C1 -- RG --> E1[Empréstimo]; A1[Agencia] -- cod_ag --> D2[Depósito]; A1 -- cod_ag --> E2[Empréstimo];
```

# Chave estrangeira

- Considere as relações:

  
A(a1, a2, a3) e B(b1, b2, a1)

Diz-se que a1 é chave estrangeira em B se a1 é chave primária em outra relação, ou seja, no caso, em A.

# DDL – Criação da Tabela Cliente

```
CREATE TABLE cliente (
 RG VARCHAR(15) NOT NULL,
 nome_cli VARCHAR(45),
 ano_nasc INTEGER,
 cidade_cli CHAR(35),
 sexo ENUM { 'M', 'F' },
 PRIMARY KEY (RG)
)
```

# DDL – Criação da Tabela Cliente



Atributos

```
CREATE TABLE cliente (
 RG VARCHAR(15) NOT NULL,
 nome_cli VARCHAR(45),
 ano_nasc INTEGER,
 cidade_cli CHAR(35),
 sexo ENUM { 'M', 'F' },

 PRIMARY KEY (RG)
)
```

# DDL – Criação da Tabela Cliente

```
CREATE TABLE cliente (
 RG VARCHAR(15) NOT NULL,
 nome_cli VARCHAR(45),
 ano_nasc INTEGER,
 cidade_cli CHAR(35),
 sexo ENUM { 'M', 'F' },
 PRIMARY KEY (RG)
)
```



Chave  
primária

# DDL – Criação da Tabela Agência

```
CREATE TABLE agencia (
 cod_ag INT(8) NOT NULL,
 nome_ag VARCHAR(35),
 cidade_ag VARCHAR(35),
 ativo NUMERIC(12,2),
 PRIMARY KEY (cod_ag)
)
```

# DDL – Criação da Tabela Depósito

```
CREATE TABLE deposito (
 num_dep INT(8) NOT NULL,
 RG VARCHAR(15),
 Saldo NUMERIC(12,2),
 cod_ag INT(8),
 PRIMARY KEY (num_dep),
 FOREIGN KEY (RG) REFERENCES cliente,
 FOREIGN KEY (cod_ag) REFERENCES agencia
)
```

# DDL – Criação da Tabela Depósito

```
CREATE TABLE deposito (
 num_dep INT(8) NOT NULL,
 RG VARCHAR(15),
 Saldo NUMERIC(12,2),
 cod_ag INT(8),
 PRIMARY KEY (num_dep),
 FOREIGN KEY (RG) REFERENCES cliente,
 FOREIGN KEY (cod_ag) REFERENCES agencia
)
```



Chave estrangeira

# DDL – Criação da Tabela Depósito

```
CREATE TABLE emprestimo (
 num_empr INT(8) NOT NULL,
 RG VARCHAR (15),
 Valor NUMERIC (12,2),
 cod_ag INT(8),
 PRIMARY KEY (num_empr),
 FOREIGN KEY (RG) REFERENCES cliente,
 FOREIGN KEY (cod_ag) REFERENCES agencia
)
```

# DDL – Remover tabela

- ❑ Remover uma tabela por completo.

```
drop table deposito
```

- ❑ Caso a tabela seja referenciada por outra é necessário remover a outra tabela primeiro.

# DDL – Alterar Tabela

- Adicionar atributo da tabela

```
alter table deposito add column data_deposito date;
```

- Remover atributo da tabela

```
alter table deposito drop column data_deposito date;
```

# SQL

## 7.3 Consulta SQL

# Consulta SQL

- A estrutura básica para realizar consultas, consiste em três cláusulas, **select**, **from** e **where**.

```
SELECT < lista de atributos >
FROM < lista de relacoes > [WHERE < predicado >]
```

# Consulta SQL

- A consulta retorna o nome de todos os clientes cadastrados na tabela Cliente

```
SELECT nome
FROM cliente
```

# Consulta SQL

- Para evitar a exibição de resultados duplicados utiliza-se o comando **distinct**.

```
SELECT DISTINCT nome_cidade,
FROM cliente
```

# Consulta SQL

- Para retornar apenas uma seleção dos dados é possível utilizar a cláusula **WHERE** para filtrar os resultados e exibir apenas aqueles de interesse.

```
SELECT nome_cli
FROM cliente
WHERE sexo = 'M' and ano_nasc = 1984
```

# Consulta SQL

- A cláusula **FROM** pode receber uma lista de relações.
- Deve ser informada todas as relações que possuem informações que deseja-se obter
- Utilizar a cláusula **WHERE** para especificar a condição para combinar as informações.

# Consulta SQL

- Neste caso é feito o produto cartesiano entre as relações.

```
SELECT cliente.nome_cli
FROM cliente,
 deposito
WHERE cliente.RG=deposito.RG
```

# Consulta SQL

- Um exemplo mais elaborado de consulta pode ser o seguinte: “Obter todos os empréstimos do cliente Fernando cujos valores superam 500 reais”

```
SELECT num_empr, valor, cod_ag

FROM cliente, emprestimo

WHERE nome_cli = 'Fernando'

 AND cliente.RG = emprestimo.RG

 AND valor > 500,00
```

# Consulta SQL - IN

- É possível aplicar operações de conjunto em uma consulta SQL, por meio do operador IN.
- Exemplo: encontrar todos os clientes com depósito superior a 500 reais

```
SELECT *
FROM cliente
WHERE RG IN
 (SELECT RG
 FROM deposito
 WHERE valor > 500)
```

# Consulta SQL - UNION

- ❑ O comando **UNION** é compatível somente se apresentar o mesmo tipo de domínio, apesar de ser composto por atributos diferentes.
- ❑ Exemplo: Todos os empréstimos e ou depósitos que ocorreram na agencia 101.

```
(SELECT num_dep,
 saldo
FROM deposito
WHERE cod_ag=101)
UNION
(SELECT num_empr,
 valor
FROM emprestimo
WHERE cod_ag=101)
```

# Consulta SQL - INTERSECT

- ❑ O comando **INTERSECT** possui o mesmo funcionamento do **UNION**.
- ❑ Exemplo: Todos os clientes que efetuaram um depósito de pelo menos 200,00 na agência 101 e também realizaram empréstimo na agência 101.

```
(SELECT RG
 FROM deposito
 WHERE cod_ag=101 and valor>200,00) INTERSECT
(SELECT RG
 FROM emprestimo
 WHERE cod_ag=101)
```

# Consulta SQL - MINUS

- ❑ O comando **MINUS** possui o mesmo funcionamento do **UNION**.
- ❑ Exemplo: Todos os RG dos clientes que possuem um depósito acima de 200,00 na agência 101 e que não possuem empréstimo de mesmo valor e agência.

```
(SELECT num_dep, saldo

FROM deposito

WHERE cod_ag = 101 AND saldo >= 200,00) MINUS

 (SELECT num_empr, valor

 FROM emprestimo

 WHERE cod_ag = 101)
```

# Consulta SQL - AS

- Conceito de variáveis tuplas – é possível atribuir a uma mesma relação, vários identificadores.

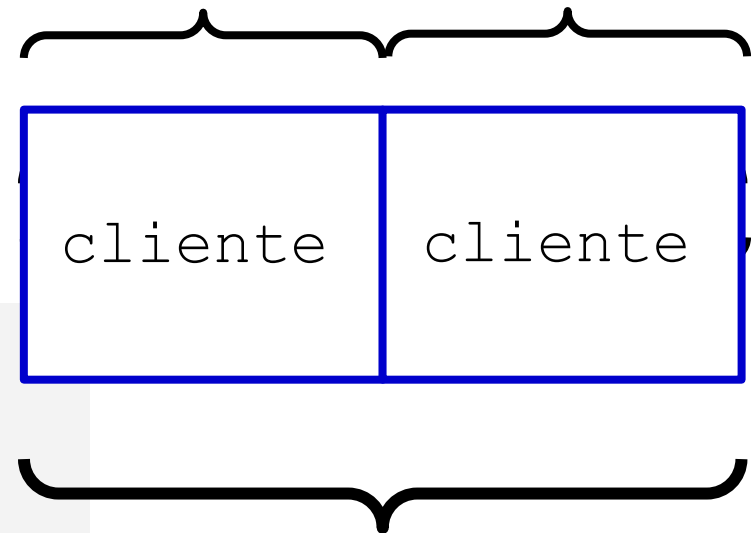
```
SELECT X.RG,
 X.nome_cli,
 X.data_nasc
FROM cliente AS X,
 cliente AS Y
WHERE Y.nome = 'Fernando'
 AND X.cidade_cli = Y.cidade_cli
```

# Consulta SQL - AS

```
SELECT X.RG,
 X.nome_cli,
 X.data_nasc
FROM cliente AS X,
 cliente AS Y
WHERE Y.nome = 'Fernando'
 AND X.cidade_cli = Y.cidade_cli
```

Tuplas de X

Tuplas de Y



Produto cartesiano

# Consulta SQL - SOME

- O operador **SOME** é utilizado para comparar valor com conjuntos.
- Consulta: “Todos os clientes que possuem empréstimos maiores que pelo menos um dos empréstimos de Pedro (RG: 3832012)”

```
SELECT DISTINCT RG
FROM deposito
WHERE saldo > SOME
 (SELECT saldo
 FROM deposito
 WHERE RG = 3832012)
```

# Consulta SQL - SOME

```
SELECT DISTINCT RG
FROM deposito
WHERE saldo > SOME
 (SELECT saldo
 FROM deposito
 WHERE RG = 3832012)
```

>, <, >=, <=, <>, =

< SOME  
= SOME  
>= SOME  
>= SOME  
<= SOME  
<> SOME

# Consulta SQL - ALL

- O operador **ALL** é utilizado para comparar valor com conjuntos.
- Consulta: O cliente mais velho por cidade

```
(SELECT x.RG, x.nome_cli, x.cidade_cli

FROM cliente AS x

WHERE data_nasc <= ALL

 (SELECT y.data_nasc

 FROM cliente As y

 WHERE x.cidade_cli=y.cidade_cli))
```

# Consulta SQL - ALL

```
(SELECT x.RG, x.nome_cli, x.cidade_cli
FROM cliente AS x
WHERE data_nasc <= ALL
 (SELECT y.data_nasc
 FROM cliente As y
 WHERE x.cidade_cli=y.cidade_cli))
```

**Comparison Operators:**  $>$ ,  $<$ ,  $>=$ ,  $<=$ ,  $<>$ ,  $=$

|      |     |
|------|-----|
| $<$  | ALL |
| $=$  | ALL |
| $>=$ | ALL |
| $>=$ | ALL |
| $<=$ | ALL |
| $<>$ | ALL |

# SQL

## 7.4 Função de Agregação

# Função de Agregação

- O SQL suporta funções de agregações, que recebem uma coleção de valores como entrada e retorna um único valor.
- O SQL possui as seguintes funções de agregação:
  - ▣ avg() – média aritmética
  - ▣ sum() – soma
  - ▣ min() – mínimo dentre os resultados obtidos
  - ▣ max() – máximo dentre os resultados obtidos
  - ▣ count() – conta o número de resultados

# Função de Agregação

- Exemplo: Total do valor solicitado de empréstimo por Fernando.

```
SELECT sum(valor) AS total_emprestimo
FROM emprestimo
WHERE RG = 3832014
```

# Função de Agregação

- ❑ **GROUP BY:** aplicar as funções de agregação para um grupo.
- ❑ Com isso tuplas com o mesmo valor do atributo especificado no GROUP BY são agrupadas

# Função de Agregação

- Exemplo: Média de empréstimos realizados considerando todas as agências de uma cidade.

```
SELECT cidade_ag,
 avg(valor)
FROM emprestimo, agencia
WHERE emprestimo.cod_ag = agencia.cod_ag
GROUP BY cidade_ag
```

# Função de Agregação

- ❑ **HAVING:** utilizado para definir um predicado que todos os grupos serão verificados.
- ❑ Aqueles que não satisfazerem a regra definida serão removidos do resultado da consulta.
- ❑ A cláusula **having** só poderá ser utilizada havendo a respectiva cláusula **group by**, ou seja, não há **having** sem **group by**.

# Função de Agregação

- Exemplo: Todos os nomes dos clientes com depósitos superiores a média das suas respectivas agências, sendo que deva ser consideradas apenas agências com média de saldo superior a 1.200

```
SELECT x.nome_cli,
 x.RG
FROM cliente x,
 deposito y
WHERE x.RG = y.RG
 AND y.saldo >= ALL
 (SELECT avg(saldo)
 FROM deposito z
 GROUP BY y.cod_ag HAVING avg(saldo) > 1.200)
```

# SQL

## 7.5 Atualizar Informações

# Inserir Informações

- Para adicionar novas tuplas em uma relação a sintaxe é:

```
INSERT INTO < relacao >
```

```
VALUES < lista de atributos e valores >
```

# Inserir Informações

- Exemplo: Passar para depósito todos os empréstimos maiores que 100 do 'João José da Silva'

```
INSERT INTO deposito
SELECT x.num_empr,
 y.RG,
 x.saldo,
 x.cod_ag
FROM emprestimo x,
 cliente y
WHERE x.RG = y.RG
 AND y.nome_cli = 'João José da Silva'
 AND x.valor >= 100
```

# Atualizar Informações

- Para atualizar uma informação já existente na tabela a sintaxe é:

```
UPDATE R

SET < expressao >

WHERE p
```

- Onde R é uma relação, < expressão > contém a atribuição contemplando o novo valor e 'p' é o predicado a ser entendido.

# Atualizar Informações

- Exemplo: Aumento de 50% nos depósitos do 'Antônio Manuel (RG: 12952) que tenham valores superiores a 1000'

```
UPDATE deposito
SET saldo = 1.5 * saldo
WHERE RG = 12952
 AND saldo > 1000
```

# Remover Informações

- Para remover uma informação da tabela a sintaxe é:

```
DELETE
FROM R [WHERE P]
```

- Onde R é uma relação e 'P' é o predicado a ser entendido.

# Remover Informações

- Exemplo: Eliminar todos os clientes que residem na mesma cidade que o cliente 'João José da Silva'

```
DELETE
FROM cliente
WHERE cidade_cli IN
 (SELECT cidade_cli
 FROM cliente
 WHERE nome_cli = 'João José da Silva')
```

# Referências Bibliográficas

# Referências Bibliográficas

- Elmasri, R.; Navathe, S. B. Sistemas de banco de dados. São Paulo: Pearson Addison Wesley, 4<sup>a</sup> ed., 2005.
- Silberschatz, A.; Korth, H. F.; Sudarshan, S. Sistema de banco de dados. Rio de Janeiro: Elsevier, 5<sup>a</sup> ed., 2006.
- Cattell, R. G. G.; Barry, D. K.; contributors Berler, M. et al. The Object Data Standard: ODMG 3.0. São Francisco: Morgan Kaufmann, 2000.
- Object Database Management Group (ODMG) Disponível em: [http://wwwasd.web.cern.ch/wwwasd/rd45/DRDCproposal/dr dc\\_17.html](http://wwwasd.web.cern.ch/wwwasd/rd45/DRDCproposal/dr dc_17.html). Acesso em 10 de Outubro de 2013.
- Raza, M. I.; Zaib, Q. J.; Farooq, M. S.; Abid, A.; Khan, S. A. Meticulous analysis of Semantic Data Model An optimal approach for ERD. Journal of Basic and Applied Scientific Research, vol. 2, num. 8, pp. 8344–8354, 2012.
- Raghu, R; Gehrke, J. Database management systems. NewYork: McGraw-Hili, 3<sup>a</sup> ed., 2003.